

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 - DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0614	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 - Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 - Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	N. SRIKANTH	Reg. No. :	192411139
Section / Batch:		Date:	

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A - DEBUGGING & OPTIMISATION TASK

Q1. Debugging Bubble Sort Code (Inner Loop Uses Fixed Upper Bound)

The following Bubble Sort implementation always runs the inner loop up to the last index, even though the largest elements have already settled at the end after each pass. Carefully analyze why this does not affect correctness in every case but still introduces inefficiency and can increase the risk of unnecessary comparisons. Debug the code to use the proper shrinking inner loop boundary. Optimize the implementation further using a swap flag to stop early when the array becomes sorted. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final version with comments.

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(0, n-1): # ISSUE  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
    return arr
```

Q2. Debugging Insertion Sort Code (Incorrect Final Placement Index)

The following Insertion Sort implementation performs element shifting, but the final insertion step uses the wrong index and causes misplaced values in the sorted prefix. Carefully analyze the shifting process and identify the exact source of the indexing error. Debug the code step-by-step so that each element is inserted into its correct position after all larger elements are shifted. Optimize the implementation for readability and maintain consistent variable usage. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean version with proper comments.

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j+1] = arr[j]  
            j -= 1  
        arr[j] = key    # ERROR  
    return arr
```

Code of Conduct

I _____ N. SRIKANTH192411139 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each - 50)
Q1	4	3	3	2	4	39
Q2	4	4	4	3	2	44
Overall Total (100)						83

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Debugging Bubble Sort Code (Inner Loop Uses Fixed Upper Bound)

Given Code

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n-1):    # ISSUE
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr
```

Identify the Errors

The error is in the inner loop boundary:

```
for j in range(0, n-1)
```

The loop always runs through the entire array during every pass. After each pass, the largest element is already moved to its correct position at the end, so comparing those elements again is unnecessary.

Why Incorrect Values Are Produced

The given code does not produce incorrect values. It correctly sorts the array. The issue is inefficiency because it performs extra comparisons even when some elements are already sorted. This increases execution time and reduces performance for large datasets.

Step-by-Step Debugging

1. Identify the fixed inner loop boundary that causes unnecessary comparisons.
2. Replace the fixed boundary:

```
range(0, n-1)
```

with a shrinking boundary:

```
range(0, n-i-1)
```

because the last i elements are already sorted after each pass.

3. Add a swap flag to check whether any swapping occurs in a pass.
4. If no swaps occur, terminate the algorithm early because the array is already sorted.

Corrected Code

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
```

```

        arr[j], arr[j+1] = arr[j+1], arr[j]
        swapped = True

    if not swapped:
        break

    return arr

```

Time Complexity

- **Best Case:** $O(n)$
 - Array is already sorted, so the algorithm stops after one pass.
- **Average Case:** $O(n^2)$
 - Random order requires multiple passes and swaps.
- **Worst Case:** $O(n^2)$
 - Reverse sorted array requires maximum comparisons.
- **Space Complexity:** $O(1)$

Final Documented Code

```

def bubble_sort(arr):
    # Get the size of the array
    n = len(arr)

    # Perform sorting passes
    for i in range(n):

        # Flag to check if swapping occurs
        swapped = False

        # Compare unsorted elements only
        for j in range(0, n-i-1):

            # Swap elements if they are in wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True

        # Stop if array is already sorted
        if not swapped:
            break

    return arr

```

Conclusion

The original Bubble Sort algorithm is correct but inefficient because it repeatedly compares elements that are already sorted. The optimized version uses a shrinking inner loop and a swap flag to reduce unnecessary comparisons and improve performance, especially for already sorted or nearly sorted arrays.

Q2. Debugging Insertion Sort Code (Incorrect Final Placement Index)

Given Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1

        arr[j] = key    # ERROR

    return arr
```

Identify the Errors

The error is in the final insertion statement:

```
arr[j] = key
```

After shifting larger elements, the variable `j` is decreased one extra position. The correct position for inserting the key is `j + 1`, not `j`.

Correct statement:

```
arr[j+1] = key
```

Why Incorrect Values Are Produced

The incorrect index places the key element one position before its actual location.

During shifting:

- Larger elements are moved one position to the right.
- The value of `j` is reduced until the correct insertion position is found.
- Since `j` stops at the index before the insertion point, the key must be placed at `j+1`.

Using `arr[j] = key` overwrites an incorrect position and causes misplaced values in the sorted portion.

Step-by-Step Debugging

Example:

Array:

```
[5, 3, 4, 1]
```

Pass 1:

```
key = 3
```

```
j = 0
```

Compare:

```
5 > 3
```

Shift 5:

```
[5, 5, 4, 1]
```

Decrease `j`:

```
j = -1
```

Correct insertion:

`arr[j+1] = arr[0] = 3`

Result:

[3, 5, 4, 1]

Pass 2:

key = 4

Shift 5:

[3, 5, 5, 1]

Insert key at:

`j+1 = 1`

Result:

[3, 4, 5, 1]

Pass 3:

key = 1

Shift all larger values and insert at the beginning.

Final result:

```

j = i - 1

# Shift larger elements one position ahead
while j >= 0 and arr[j] > key:
    arr[j + 1] = arr[j]
    j -= 1

# Insert key at the correct position
arr[j + 1] = key

return arr

```

Example `arr = [5, 3, 4, 1]`
`print(insertion_sort(arr))`

Output:

[1, 3, 4, 5]

Conclusion

The error in the original Insertion Sort code was caused by inserting the key at index `j` instead of `j+1`. After shifting larger elements, `j` points to the position before the correct insertion location. Correcting the index ensures that each element is placed properly in the sorted prefix. The optimized implementation is simple, readable, and maintains $O(n)$ best-case and $O(n^2)$ average and worst-case time complexity.